



LINGUAGEM DE PROGRAMAÇÃO ORIENTADA A OBJETOS C++

Encapsulamento Método Construtor

Visibilidade

- **private:** visível apenas dentro da classe.
- **protected:** visível apenas dentro da classe e dentro das subclasses (hierarquia).
- **public:** visível por todas as classes.

Aplicados a atributos e métodos.

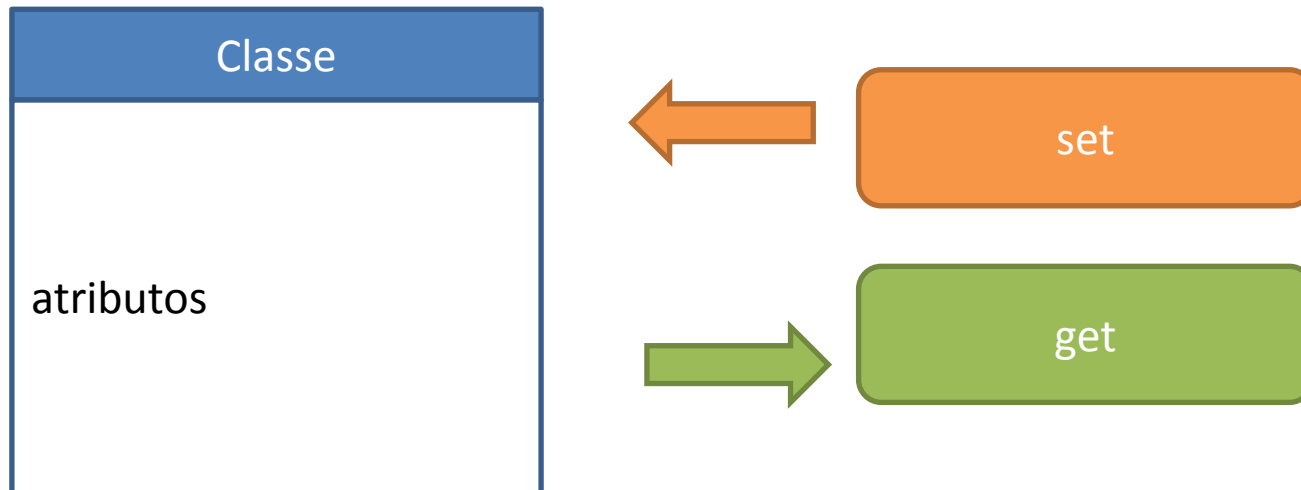
Encapsulamento

Por que controlar o acesso com encapsulamento?

- Protege informação privada dentro da classe.
- Esclarece através de métodos de acesso como outros programadores devem usar sua classe.
- Separar a implementação da interface.

Métodos de Acesso (métodos acessórios)

- Não confundir com assessor.



Exemplo (**não encapsulado**)

```
class Pessoa
{
    public:
        int registro;
        string nome;
        string contaCorrente; //número da conta
}
```

Exemplo (**encapsulado**)

```
class Pessoa
{
    private:
        int registro;
        string nome;
        string contaCorrente; //número da conta
}
```

Encapsulamento

- Para cada campo encapsulado, ou seja, mudar visibilidade pública para privada, deve gerar um método público SET e um método público GET.
- Exemplo:
`private:`
`int registro;`

Método GET

O método GET retorna o valor armazenado no campo **registro**.

O método tem retorno e não tem parâmetro!

public:

```
int getRegistro()  
{  
    return registro;  
}
```

Método SET

O método SET não retorna o valor armazenado no campo **registro**, mas permite através do parâmetro atualizar o valor armazenado no atributo.

O método não tem retorno, mas tem parâmetro!

public:

```
void setRegistro(int val)
{
    registro = val;
}
```

```
1 class Pessoa
2 {
3     private:
4         int registro;
5         string nome;
6         string contaCorrente; //número da conta
7
8     public:
9         void setRegistro(int val)
10        {
11            registro = val;
12        }
13
14        int getRegistro()
15        {
16            return registro;
17        }
18
19 }
```

Construtor

- A linguagem C++ é concebida para fazer tipos definidos pelo usuário (classes). Permite uma inicialização direta dos objetos;
- O compilador na criação de um objeto chama o método construtor de uma classe;
- Os construtores são muito úteis em tarefas ligadas à inicialização de objetos. Nelas pode-se encontrar inicialização direta de membros, alocação dinâmica de memória, recuperação de dados em arquivos, etc.
- O construtor é uma função com o mesmo nome da classe e que não pode retornar nenhum valor.

Construtor

- Método que é executado na construção de objetos da classe;
- Não tem retorno;
- Construtor possui o mesmo nome da classe;
- O método deve ser público;
- Quando o construtor não é fornecido pelo programador, é gerado um construtor padrão (sem parâmetros) automático;
- Construtores podem receber parâmetros da mesma forma que métodos;

Exemplo

```
class Pessoa
{
    private:
        int registro;
        string nome;
        string contaCorrente;

    public:
        Pessoa (int _registro, string _nome, string _contaCorrente)
        {
            registro = _registro;
            nome = _nome;
            contaCorrente = _contaCorrente;
        }
        Pessoa ()
        {

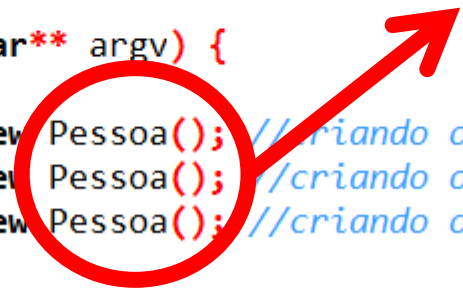
        }

}
```

Usando o exemplo da Semana 4

```
1  #include <iostream>
2  #include <string>
3  #include "Pessoa.cpp"
4
5  using namespace std;
6
7  int main(int argc, char** argv) {
8
9      Pessoa *pessoa1 = new Pessoa(); //criando objetos
10     Pessoa *pessoa2 = new Pessoa(); //criando objetos
11     Pessoa *pessoa3 = new Pessoa(); //criando objetos
12
13     pessoa1->nome = "Juquinha";
14     pessoa1->idade = 28;
15     pessoa1->altura = 1.67;
16
17     pessoa2->nome = "Michelle";
18
19     cout<< "Nome Pessoa: " << pessoa1->nome << endl;
20     cout<< "Idade Pessoa: " << pessoa1->idade << endl;
21     cout<< "Altura Pessoa: " << pessoa1->altura << endl;
22
23     return 0;
24 }
```

Chamada do método construtor!!!



Usando o exemplo da Semana 4

```
1  #include<string>
2
3  using namespace std;
4
5  class Pessoa{
6
7      public:
8          string nome;
9          int idade;
10         double altura;
11
12     };
```

Tem
método
construtor
nesta
Classe?

Esta classe tem
encapsulamento?

Usando o exemplo da Semana 4 com encapsulamento e método construtor com parâmetros e inicializações

```
1  #include<string>
2  using namespace std;
3  class Pessoa {
4
5  private:
6      string nome;
7      int idade;
8      double altura;
9
10 public:
11     Pessoa(string _nome, int _idade, double _altura){
12         nome = _nome;
13         idade = _idade;
14         altura = _altura;
15     }
16
17     Pessoa() {
18     }
19
20     void setNome (string _nome) {
21         nome = _nome;
22     }
23
24     string getNome () {
25         return nome;
26     }
27
28     void setIdade (int _idade) {
29         idade = _idade;
30     }
31
32     int getIdade () {
33         return idade;
34     }
35
36     void setAltura (double _altura) {
37         altura = _altura;
38     }
39
40     double getAltura () {
41         return altura;
42     }
43 };
```

```
1  #include <iostream>
2  #include <string>
3  #include "Pessoa.cpp"
4
5  using namespace std;
6
7  int main(int argc, char** argv) {
8
9      Pessoa *pessoa1 = new Pessoa("Juquinha",28,1.67); //criando
10     Pessoa *pessoa2 = new Pessoa(); //criando objetos
11
12     pessoa2->setNome("Michelle");
13     pessoa2->setIdade(18);
14     pessoa2->setAltura(1.69);
15
16     cout<< "Nome Pessoa: " << pessoa1->getNome() << endl;
17     cout<< "Idade Pessoa: " << pessoa1->getIdade() << endl;
18     cout<< "Altura Pessoa: " << pessoa1->getAltura() << endl;
19
20     cout<< "Nome Pessoa: " << pessoa2->getNome() << endl;
21     cout<< "Idade Pessoa: " << pessoa2->getIdade() << endl;
22     cout<< "Altura Pessoa: " << pessoa2->getAltura() << endl;
23
24     return 0;
25 }
```

```
Nome Pessoa: Juquinha
Idade Pessoa: 28
Altura Pessoa: 1.67
Nome Pessoa: Michelle
Idade Pessoa: 18
Altura Pessoa: 1.69
```



**“Aprender é a única coisa de que a mente nunca se cansa,
nunca tem medo e nunca se arrepende”
Leonardo da Vinci**